Open Access | https://doi.org/10.25080/avte6646



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

25 Years of Interactive Scientific Computing: From IPython and Jupyter to Open Source IDE-Native Notebooks

Rodrigo Silva Ferreira[1]

[1]Independent Researcher, Pittsburgh, PA, USA

Abstract

Interactive computing has reshaped how scientists explore data, develop methods, and communicate results. This paper traces twenty-five years of that transformation, from the 2001 release of IPython through the formation of Project Jupyter to the contemporary movement that embeds notebooks inside full software-development environments. We argue that this history is best understood not as the story of a file format or a user interface, but as the steady reduction of friction between four activities that were once pursued with separate tools: exploratory analysis, software engineering, collaboration, and reproducible publication. Read this way, each generation inherited the strengths of the previous one while absorbing a capability that had previously lived elsewhere. We further argue that openness (*e.g.,* of source code, of standards such as the Jupyter messaging protocol and the nbformat document schema, of community governance, and of the executable publication itself) has been the decisive enabler at every stage. After surveying the pre-Jupyter landscape and the scientific Python ecosystem that grew alongside these tools, we examine the well-documented limitations of traditional notebooks, the engineering practices that IDE-native notebooks restore, and the opportunities and risks introduced by large language models. The paper is itself an executable Jupyter notebook: every figure and quantitative claim is produced by the code it contains.

Keywords interactive computing, Jupyter, IPython, reproducible research, scientific Python, literate programming, open science, notebooks

1. Introduction

Computation has become a third mode of scientific inquiry, standing beside theory and experiment rather than merely serving them. Climate projections, genome assemblies, gravitational-wave detections, and the calibration of language models are all, at bottom, programs run over data. As the centre of gravity of research has shifted toward software, the tools used to write that software have stopped being incidental laboratory equipment and have become part of the scientific method itself. How a result is computed (and whether that computation can be inspected, rerun, and trusted by others) is now inseparable from whether the result is believed.

This shift has placed *reproducibility* at the heart of methodological debate. D. L. Donoho [1] framed reproducible computational research as the discipline of publishing, alongside each figure and table, the complete software environment that generated it, reviving a tradition that J. F. Claerbout and M. Karrenbach [2] and J. B. Buckheit and D. L. Donoho [3] had pioneered in geophysics and signal processing. A decade later, an interdisciplinary consensus statement in *Science* called for code and data to be treated as first-class research artifacts [4], and community guides distilled the accumulated experience into actionable

Published Jul 13, 2026

Correspondence to
Rodrigo Silva Ferreira
rodrigosf672@gmail.com

Open Access

Copyright © 2026 Ferreira. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

practice [5], [6]. The recurring lesson is that scientific software is rarely written the way production software is. It grows through *exploratory programming*: a tight loop of hypothesis, quick implementation, visualization, and revision, in which the program and the scientist's understanding co-evolve.

Interactivity changed research because it changed the unit of iteration. When a computation must be edited, recompiled, and resubmitted as a batch job, the cost of a single question is measured in minutes or hours, and the scientist learns to ask few, large questions. When the same computation answers in the time it takes to press a key, the cost of a question falls toward zero, and a different epistemic style becomes possible: a hypothesis is posed, tested against data, visualized, and revised dozens of times in an afternoon. The interactive loop does not merely speed up analysis; it widens the space of analyses a researcher will attempt, surfacing patterns and mistakes that a batch workflow would never have prompted anyone to look for. This is why cheap interactivity, far from being a mere convenience, reorganized the practice of computational science around exploration, and why the tools that host that exploration came to matter so much.

Exploration and communication, however, pull in opposite directions. Exploration rewards speed, mutable state, and disposable code; communication rewards clarity, structure, and permanence. D. E. Knuth [7]'s vision of *literate programming* (*i.e.,* a program written as a human narrative, with source code woven into prose) was an early attempt to reconcile the two, prioritizing explanation to people over instruction to machines. Interactive computing environments inherited this tension and, over twenty-five years, progressively dissolved it. The thesis of this paper is that the history of interactive scientific computing is the history of *reducing friction* among four activities (*i.e.,* exploration, engineering, collaboration, and reproducible publication) and that *openness* in its several forms has been the mechanism by which that friction was removed. Figure 1 previews the milestones along the way; the sections that follow trace them in turn.

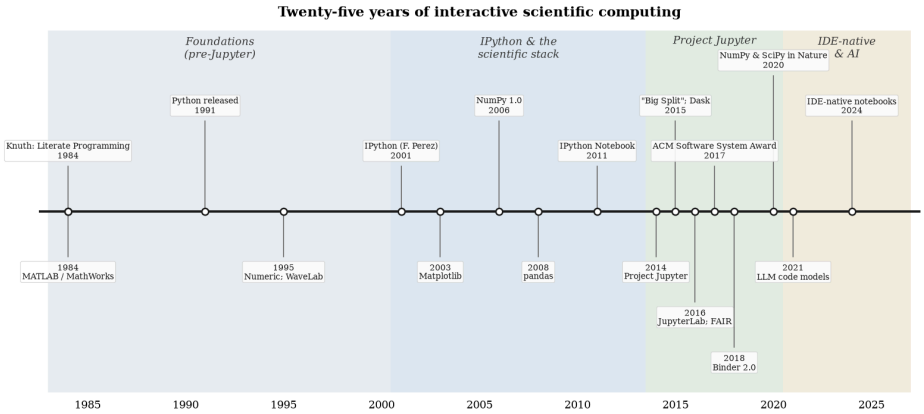


Figure 1. *A timeline of interactive scientific computing, organized into four overlapping eras. Each era inherits the capabilities of the previous one while integrating a concern (e.g., rich output, language independence, reproducible environments, software engineering) that had previously required separate tools. Dates are drawn from primary sources cited throughout the paper.*

2. Before Jupyter: REPLs, computer algebra, and literate programming

Interactivity in computing is older than the personal computer. Its lineage runs back to Lisp, whose read–eval–print loop (REPL) let a programmer type an expression and see its value immediately; McCarthy's language of 1958–1960 and the interactive Lisp systems built on it established a style of conversation with a running process that every later environment

would imitate. For decades, however, the dominant mode of scientific computing was the opposite of conversational. Programs were compiled, submitted as batch jobs, and inspected only after they finished (a workflow well matched to expensive, shared mainframes but hostile to the rapid, speculative iteration that data analysis demands).

The first tools to make scientific computing genuinely interactive were, tellingly, commercial. MATLAB began in the late 1970s when Cleve Moler wrote a Fortran "matrix laboratory" so that students could call the LINPACK and EISPACK linear-algebra libraries without writing Fortran themselves; he used it during a 1979 sabbatical at Stanford, and in 1984 he and Jack Little founded MathWorks to commercialize it [8]. Computer-algebra systems followed a parallel path: Maple was conceived at the University of Waterloo in late 1980 as a lower-cost alternative to running Macsyma, and Mathematica was launched by Wolfram Research on 23 June 1988, introducing a *notebook* interface in which cells of evaluatable input and formatted output were interleaved on a single page. The notebook metaphor that Jupyter would later make ubiquitous was thus already two decades old. Statistical computing, meanwhile, was transformed by the S language developed at Bell Labs by Becker, Chambers, and colleagues [9], whose open-source reimplementation as R by R. Ihaka and R. Gentleman [10] would become the lingua franca of statistics.

These systems were powerful, but each was, in a precise sense, *closed*. Most were proprietary; each was wedded to a single language and a single vendor; and the documents they produced (where documents existed at all) were rarely separable from the application that created them. In parallel, a quieter intellectual movement was preparing the ground for something more open. Knuth [7], [11] argued in *literate programming* that a program should be written for human readers, as an essay whose narrative order need not match the compiler's, with code extracted ("tangled") and documentation typeset ("woven") from one source; his WEB system, built to document the TeX typesetting program, was the proof of concept. At nearly the same moment, J. F. Claerbout and M. Karrenbach [2] coined the term *reproducible research* for the practice of bundling, with a publication, the electronic documents and code needed to regenerate every figure, and J. B. Buckheit and D. L. Donoho [3] popularized the idea in the WaveLab toolbox, distributing the exact code that produced each plot in their wavelet papers. Literate programming supplied the *form* (*i.e.*, narrative and code as one document) and reproducible research supplied the *purpose*. Interactive scientific computing in the Python era would inherit both, and add what the earlier systems lacked: an open-source implementation, an open document standard, and a community free to extend them.

3. IPython: an open, interactive shell for scientific Python

In 2001, Fernando Pérez was a graduate student in physics at the University of Colorado, Boulder, who wanted for Python the conveniences he admired in Mathematica and Maple: numbered input and output prompts, easy access to previous results, on-the-spot help and introspection, and a configurable, comfortable interactive session. What he wrote, as he later described it, was a 259-line "thesis procrastination project" (*i.e.*, a script loaded at startup) that gave the standard Python prompt a memory and a personality. Released as IPython 0.0.1 in December 2001 [12], it soon merged ideas and code from two kindred efforts, Janko Hauser's IPP and Nathaniel Gray's LazyPython, combining a well-organized architecture with terse interactive syntax and rich, colored tracebacks (IPython development team; [13]).

The design choices that followed proved durable. IPython introduced *magic commands* (e.g., line magics such as `%timeit` and `%matplotlib`, and cell magics prefixed with `%%`) that extend the language of the session without polluting the Python namespace. It made *introspection* immediate: appending `?` to any object prints its documentation and signature, and `??` shows

its source. It cached inputs and outputs so that prior results remained addressable, and it offered tab completion and a configuration system that let users shape their environment. Crucially, and unlike the commercial systems that inspired it, IPython was open source from its first day, developed in the open by a widening circle of contributors and eventually sustained by foundation grants (*e.g.*, an Alfred P. Sloan Foundation award in 2013, followed in 2015 by roughly six million dollars from the Helmsley, Moore, and Sloan foundations to expand the project) [13], [14]. The small convenience of a better prompt had become community infrastructure.

The demonstration below uses one of those magics. The kernel executing this very notebook is IPython, so `%timeit` is available to us as we read; it quantifies the single most important habit in scientific Python (*i.e.*, replacing explicit Python loops with vectorized array operations), a point we return to in The scientific Python ecosystem, accelerated.

```
37.9 ms ± 1.26 ms per loop (mean ± std. dev. of 3 runs, 3 loops each)
124 µs ± 14 µs per loop (mean ± std. dev. of 5 runs, 50 loops each)

Vectorization speed-up on this machine: 350x
```

The two lines compute the same thing (*i.e.*, the element-wise square of a million numbers) but the vectorized form runs hundreds of times faster. Being able to *measure* that gap in a single keystroke, mid-thought, is the essence of what IPython added: not a new language, but a new tightness in the loop between writing code and understanding its behaviour.

4. Project Jupyter: from a shell to an open protocol

IPython's most consequential idea was architectural rather than cosmetic. To support a graphical Qt console (2011) and then a browser-based notebook (first shipped in IPython 0.12 in December 2011, drawing heavy inspiration from William Stein's pioneering Sage Notebook), the project separated the *frontend* that a user interacts with from the *kernel* that executes code, with the two communicating over a well-specified network protocol. Once execution lived behind a protocol rather than inside the shell, the language of the kernel became an implementation detail. A frontend that could talk to a Python kernel could talk to any kernel.

This realization motivated "the Big Split." IPython 3.0, released in February 2015, was the last release to bundle the language-agnostic notebook with the Python-specific shell; IPython 4.0, on 11 August 2015, moved the frontend, the notebook document format, and the protocol into a new, language-neutral umbrella: Project Jupyter, whose name honours its three founding languages (*i.e.*, Julia, Python, and R [10]) and alludes to Galileo's notebooks recording the moons of Jupiter. This transition, formalized in a defining 2016 paper authored by core architects Thomas Kluyver, Min Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Damian Avila, Kyle Kelley, Carol Willing, Jason Grout, Sylvain Corlay, and others [15], was not a rebranding but a deliberate act of *opening*: it declared that the value of the notebook lay in a set of open standards that no single language community owned.

Figure 2 sketches the resulting architecture. A frontend and a kernel communicate over a ZeroMQ-based messaging protocol [16] carried on five sockets (*i.e.*, Shell, IOPub, stdin, Control, and Heartbeat), each with a defined role (Jupyter development team, *Messaging*). The Shell channel carries `execute_request` messages and their replies; the IOPub channel broadcasts side effects (*e.g.*, standard output, rich `display_data`, and `execute_result`) to every connected client; stdin handles input prompts; Control carries high-priority interrupt and shutdown messages; and Heartbeat confirms liveness. Every message is signed with an

HMAC digest and serialized, by default, as JSON, so that any process implementing the protocol can serve as a kernel.

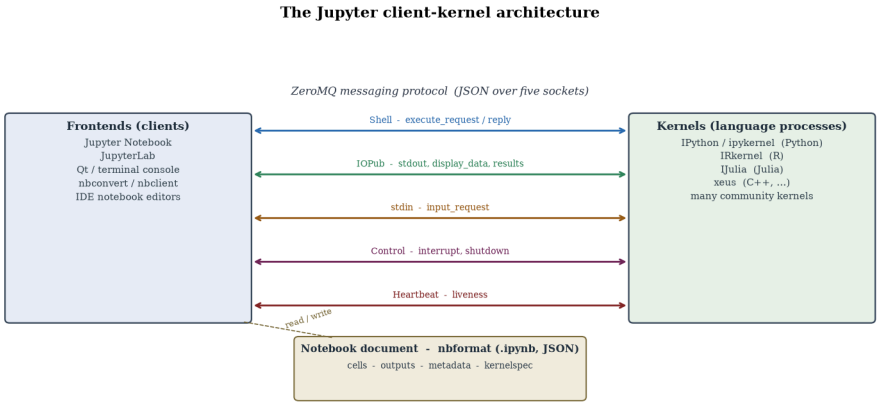


Figure 2. *The Jupyter client–kernel architecture. Decoupling the user-facing frontend from the language kernel behind an open, signed messaging protocol is what makes Jupyter language-agnostic: a* kernelspec *registers any conforming interpreter, and dozens of community kernels now exist. The notebook document (`nbformat`) is an independent JSON artifact that frontends read and write.*

The flow of a single execution is worth making concrete, because it explains both the power and the hazards of notebooks discussed later. Figure 3 shows the message exchange when a user runs one cell.

```
sequenceDiagram
    autonumber
    participant F as Frontend
    participant S as Shell channel
    participant K as Kernel
    participant IO as IOPub channel
    F->>S: execute_request (source, msg_id)
    S->>K: deliver request
    K->>IO: status = busy
    K->>IO: execute_input (echo source, count = n)
    K->>IO: stream / display_data / execute_result
    K->>S: execute_reply (status = ok, count = n)
    K->>IO: status = idle
```

Figure 3. *The messaging sequence for executing one cell. The kernel maintains an execution counter `n` and a persistent namespace; the frontend records each cell's outputs and that counter in the notebook document. Because the counter reflects the* order of execution*, not the order of cells on the page, a saved notebook can encode a history that no top-to-bottom run would reproduce (*i.e.*, the seed of the reproducibility problems examined in The challenges of traditional notebooks).*

Two open standards complete the picture. The *kernelspec* mechanism (*i.e.*, a small `kernel.json` file in a known location) lets any language that speaks the protocol be installed as a kernel, which is why Jupyter today drives Python, R, Julia, C++, and many other languages from one interface. The `nbformat` schema defines the notebook itself as a JSON document with top-level `cells`, `metadata`, and version fields, where each cell is `code`, `markdown`, or `raw` and code cells carry their outputs inline [15], [17]. Because the format is open and text-based, notebooks can be generated, transformed, executed, and rendered by an entire ecosystem of tools rather than a single application. On this foundation grew JupyterLab (unveiled in alpha at SciPy 2016 and released as 1.0 in June 2019) and JupyterHub for multi-user deployments, work recognized by the 2017 ACM Software System Award, presented in 2018 [18]. The recurring pattern is openness compounding: an open protocol enabled an open document format, which enabled an open ecosystem.

5. The scientific Python ecosystem, accelerated

Jupyter did not create the scientific Python ecosystem, but it gave that ecosystem an interactive interface of unprecedented reach, and the two grew together. The stack is layered (Figure 4). At its base sits CPython with compiled extensions; above that, NumPy provides the n-dimensional array, its universal functions, and the broadcasting rules that let array expressions stand in for explicit loops [19]. NumPy traces its lineage to Numeric (Jim Hugunin, 1995) and numarray, which Travis Oliphant, Charles R. Harris, Nathaniel J. Smith, and others unified and grew into NumPy, released as 1.0 in 2006. On the array core rest the libraries most analyses touch directly: SciPy for numerical algorithms (driven by Pauli Virtanen, Ralf Gommers, Stefan van der Walt, and a massive community) [20], pandas for labeled, heterogeneous, tabular and time-series data [21], Matplotlib for publication-quality graphics [22], and SymPy for symbolic mathematics [23]. Domain libraries (*e.g.*, scikit-learn for machine learning [24], xarray for labeled N-dimensional data [25], and others) build on these, while Dask extends the same array and dataframe interfaces to larger-than-memory and parallel computation [26].

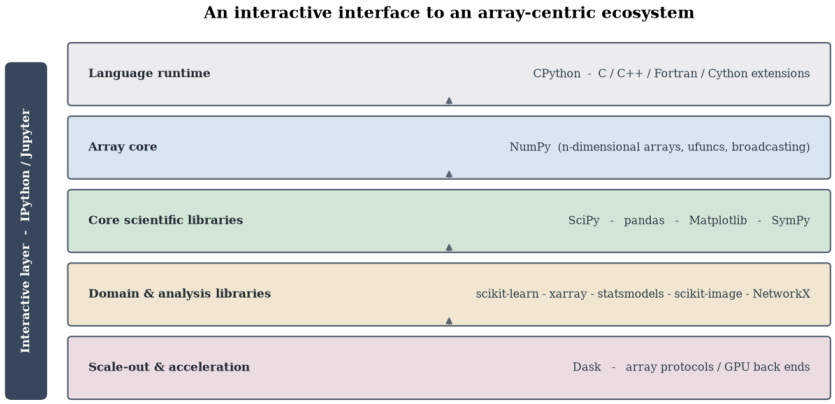


Figure 4. *The array-centric scientific Python stack. A shared array substrate (NumPy) lets independently developed libraries interoperate, and a common interactive layer (IPython/Jupyter) lets a researcher move fluidly among them. The ecosystem's coherence is a direct dividend of open, shared interfaces.*

The remainder of this section is a short tour of that stack in action, with each computation chosen to make a point rather than to decorate the page. We begin with the idiom that defines numerical Python, vectorization (already previewed in IPython: an open, interactive shell for scientific Python, now measured across problem sizes).

```
import timeit
import numpy as np
import matplotlib.pyplot as plt

sizes = [10_000, 100_000, 1_000_000]
loop_t, vec_t = [], []
for n in sizes:
    a = rng.standard_normal(n)
    loop_t.append(timeit.timeit(lambda: [v * v for v in a], number=3) / 3)
    vec_t.append(timeit.timeit(lambda: a * a, number=50) / 50)

speedup = [l / v for l, v in zip(loop_t, vec_t)]
fig, ax = plt.subplots(figsize=(6.6, 4.0))
idx = np.arange(len(sizes))
ax.bar(idx - 0.19, loop_t, 0.36, color=TOL[2], label="Python loop")
ax.bar(idx + 0.19, vec_t, 0.36, color=TOL[0], label="NumPy vectorized")
ax.set_yscale("log"); ax.set_xticks(idx); ax.set_xticklabels([f"{n:,}" for n in sizes])
ax.set_xlabel("array length"); ax.set_ylabel("seconds per call (log scale)"); ax.legend()
ax.set_title(f"Vectorization: up to {max(speedup):.0f}x faster on this machine")
_save(fig, "demo_numpy")
print("speed-ups by size:", [f"{s:.0f}x" for s in speedup])
```

```
speed-ups by size: ['58x', '272x', '254x']
```

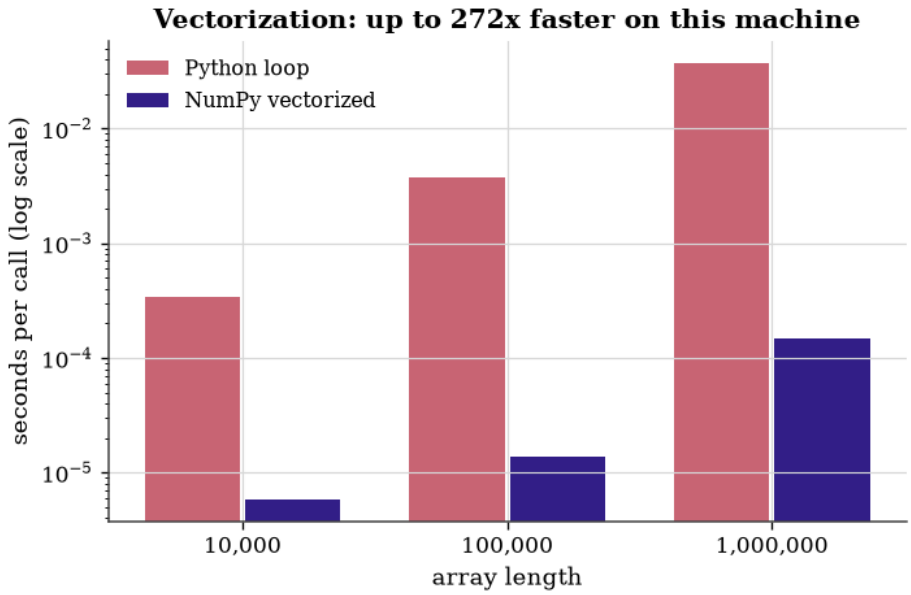


Figure 5. *Element-wise squaring via an explicit Python loop versus a NumPy array expression, timed across three problem sizes (note the logarithmic axis). The vectorized version is one to two orders of magnitude faster at every size tested, though the margin fluctuates with system load rather than widening monotonically; pushing the loop into compiled, vectorized code is the foundation on which every higher library is built.*

Real data is rarely an anonymous array; it carries labels, units, and time. pandas exists to keep those labels attached through every operation. The next cell synthesizes an hourly temperature record with a diurnal cycle and noise, then uses a `DatetimeIndex` to resample to daily means and to compute a rolling trend (operations that would be error-prone with bare arrays but are one line each with labeled data).

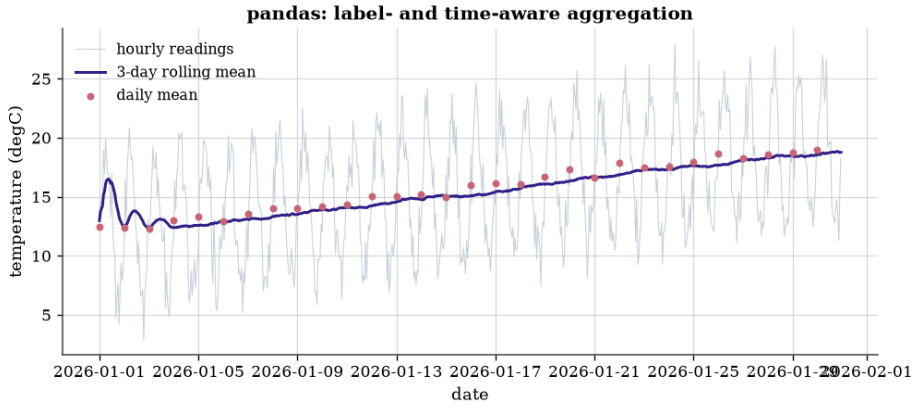
```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt

hours = pd.date_range("2026-01-01", periods=24 * 30, freq="h")
t = np.arange(len(hours))
signal = (12 + 6 * np.sin(2 * np.pi * t / 24)           # diurnal cycle
          + 0.01 * t                                     # slow warming trend
          + rng.normal(0, 1.5, len(hours)))              # measurement noise
ts = pd.Series(signal, index=hours, name="temperature_C")

daily = ts.resample("D").mean()                          # label-aware aggregation
trend = ts.rolling("3D").mean()                          # 3-day rolling mean

fig, ax = plt.subplots(figsize=(9.5, 3.8))
ax.plot(ts.index, ts.values, color="#c7d2dc", lw=0.6, label="hourly readings")
ax.plot(trend.index, trend.values, color=TOL[0], lw=1.8, label="3-day rolling mean")
ax.scatter(daily.index, daily.values, color=TOL[2], s=14, zorder=5, label="daily mean")
ax.set_xlabel("date"); ax.set_ylabel("temperature (degC)"); ax.legend(loc="upper left")
ax.set_title("pandas: label- and time-aware aggregation")
_save(fig, "demo_pandas")
print(f"hourly points: {len(ts)} | daily means: {len(daily)} | "
      f"overall mean: {ts.mean():.2f} degC")
```

```
hourly points: 720 | daily means: 30 | overall mean: 15.66 degC
```



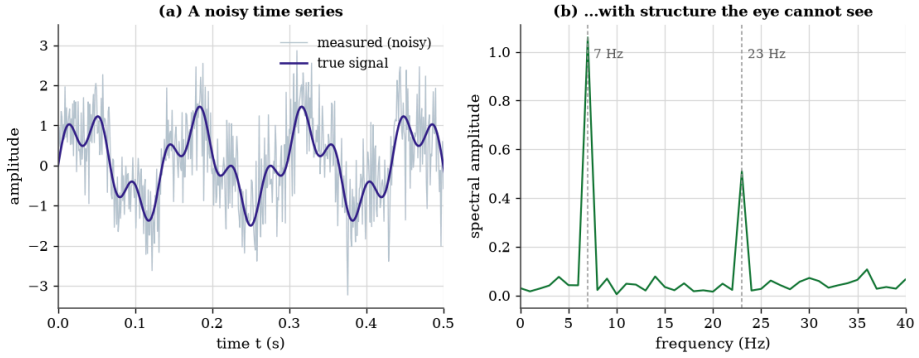
```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 1.0, 1000, endpoint=False)
clean = np.sin(2 * np.pi * 7 * t) + 0.5 * np.sin(2 * np.pi * 23 * t)
noisy = clean + 0.8 * rng.standard_normal(t.size)

freqs = np.fft.rfftfreq(t.size, d=t[1] - t[0])
amp = np.abs(np.fft.rfft(noisy)) * 2 / t.size

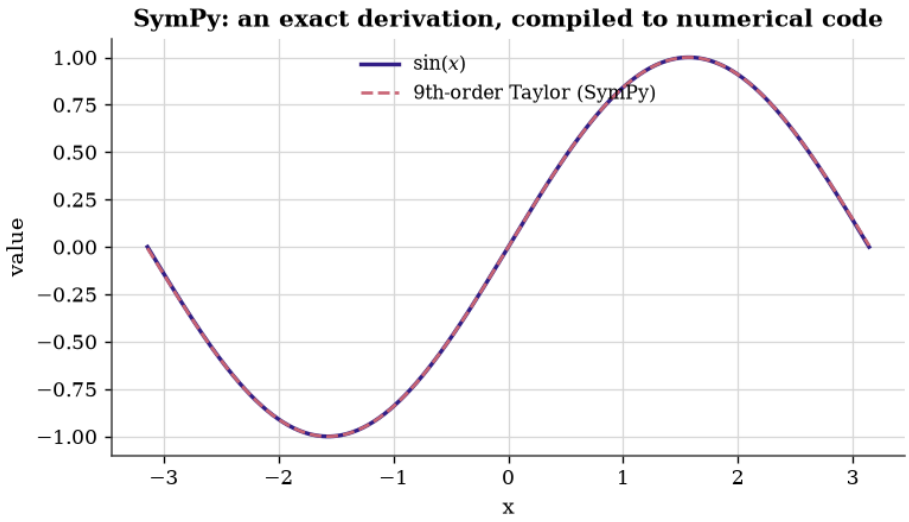
fig, ax = plt.subplots(1, 2, figsize=(11, 3.7))
ax[0].plot(t, noisy, color="#b8c4d0", lw=0.8, label="measured (noisy)")
ax[0].plot(t, clean, color=TOL[0], lw=1.6, label="true signal")
ax[0].set(xlabel="time t (s)", ylabel="amplitude", xlim=(0, 0.5))
ax[0].legend(loc="upper right"); ax[0].set_title("(a) A noisy time series", fontsize=11)
ax[1].plot(freqs, amp, color=TOL[1], lw=1.3)
for f0 in (7, 23):
    ax[1].axvline(f0, color="#999", ls=(0, (3, 2)), lw=0.9)
    ax[1].text(f0 + 0.6, amp.max() * 0.92, f"{f0} Hz", fontsize=9, color="#555")
ax[1].set(xlabel="frequency (Hz)", ylabel="spectral amplitude", xlim=(0, 40))
ax[1].set_title("(b) ...with structure the eye cannot see", fontsize=11)
_save(fig, "demo_scipy")
peaks = np.sort(freqs[np.argsort(amp)[-2:]])
print(f"recovered dominant frequencies: {peaks[0]:.1f} Hz and {peaks[1]:.1f} Hz")
```

```
recovered dominant frequencies: 7.0 Hz and 23.0 Hz
```



```
Taylor series of sin(x) to 9th order:
```

```
x**9/362880 - x**7/5040 + x**5/120 - x**3/6 + x
```




```python
import numpy as np, xarray as xr
import dask.array as da

# xarray: an array that knows what its axes mean
temps = xr.DataArray(
    rng.normal(15, 5, size=(3, 4)),
    dims=("city", "month"),
    coords={"city": ["Oslo", "Nairobi", "Lima"],
            "month": ["Mar", "Apr", "May", "Jun"]},
    name="temperature_C",
)
print("Per-city mean temperature (named-dimension reduction):")
print(temps.mean(dim="month").to_series().round(2).to_string())

# Dask: the NumPy interface, evaluated lazily and in chunks
big = da.random.random((20_000, 20_000), chunks=(2_000, 2_000))   # ~3 GB if realized
lazy_mean = big.mean()                        # nothing computed yet -> a task graph
print(f"\nDask array: {big.shape} in {big.npartitions} chunks; "
      f"graph builds instantly, mean computes to {float(lazy_mean.compute()):.4f}")
```

```
Per-city mean temperature (named-dimension reduction):
city
Oslo       17.19
Nairobi    14.04
Lima       15.68

Dask array: (20000, 20000) in 100 chunks; graph builds instantly, mean computes to 0.5000
```

Taken together, these libraries share two properties that recur throughout this paper: they are open source, and they agree on interfaces. The array protocol lets independently authored packages compose; the estimator and labeled-array conventions let a researcher carry intuition from one library to the next. Jupyter's contribution was to place a single, interactive, narrative surface over all of them. The interactivity is not merely a convenience for the author. Because a notebook can embed live controls, a reader can manipulate a computation directly, turning a static figure into an instrument. The widget below lets one

vary a sample size and a distribution and watch an empirical histogram converge toward its theoretical density.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from ipywidgets import interact, IntSlider, Dropdown

DISTS = {"normal": stats.norm(0, 1),
         "exponential": stats.expon(scale=1.0),
         "uniform": stats.uniform(-2, 4)}

def explore(n=400, dist="normal"):
    d = DISTS[dist]
    samples = d.rvs(size=n, random_state=np.random.default_rng(SEED))
    fig, ax = plt.subplots(figsize=(7.2, 3.8))
    ax.hist(samples, bins=40, density=True, color="#cdd9e3",
            edgecolor="white", label=f"{n} samples")
    grid = np.linspace(samples.min(), samples.max(), 300)
    ax.plot(grid, d.pdf(grid), color=TOL[2], lw=2.2, label="true density")
    ax.set_title(f"Empirical vs. theoretical density  (n = {n}, {dist})")
    ax.set_xlabel("value"); ax.set_ylabel("density"); ax.legend(loc="upper right")
    plt.show()

interact(explore,
         n=IntSlider(min=20, max=5000, step=20, value=400, description="samples"),
         dist=Dropdown(options=list(DISTS), value="normal", description="distribution"));
```

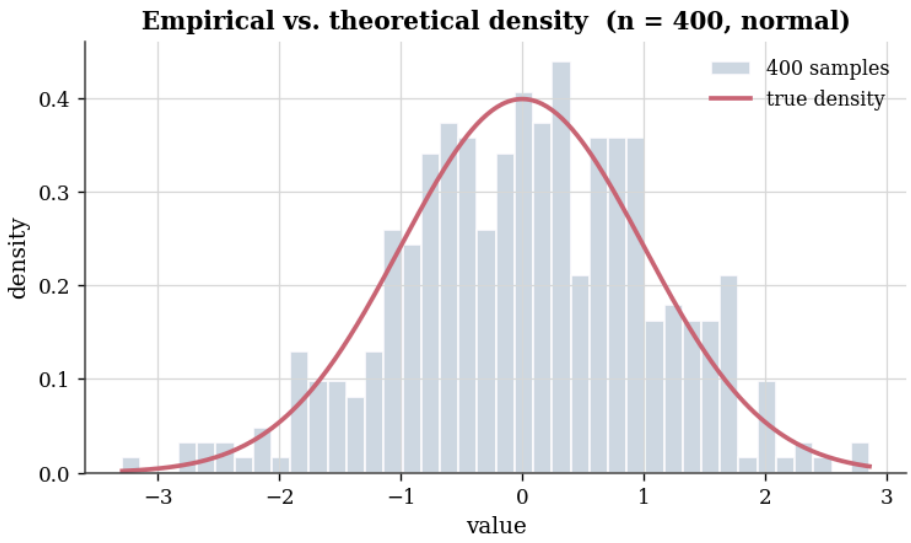


Figure 6. *A live control over sample size and distribution. In a running kernel the sliders update the figure continuously; in a static rendering, the initial state is shown. Interactivity of this kind turns a reader into an experimenter, which is precisely why notebooks became the medium of choice for teaching, exploration, and (increasingly) publication [14].*

6. Open Science and Reproducibility

If interactivity made notebooks the natural home for exploration, openness made them a candidate vehicle for reproducible publication. The two agendas converged in the mid-2010s. The FAIR Guiding Principles [27] argued that scholarly outputs should be Findable, Accessible, Interoperable, and Reusable (principles later extended explicitly to research software [28]). Practical guidance accumulated in parallel, drawing heavily on notebook pedagogy championed by educators (including Jessica Hamrick, Lorena A. Barba, Amy J. Ko, and Allen Downey) [29], [30]: the “ten simple rules” for reproducible computa-

tional research [31] and the “best practices” and “good enough practices” series [5], [6] (championed by open science leaders like Greg Wilson, Neil Chue Hong, Titus Brown, Daniel S. Katz, and Karthik Ram) translated the aspiration into habits (*e.g.*, version everything, automate the pipeline, record the environment).

Recording the environment proved to be the crux. A notebook captures code and narrative, but its results depend on a web of library versions and system libraries that the document itself does not pin. Two technologies addressed this gap. Containerization, popularized by Docker [32], let an entire software environment be specified declaratively and rebuilt identically elsewhere. Binder, pioneered by community leaders such as Yuvi Panda, Chris Holdgraf, and Tim Head [33], connected that capability to notebooks: given only a public Git repository with its dependencies declared, the `repo2docker` tool builds a container image and JupyterHub serves a live, executable session in the browser, so that a reader can run a paper’s notebooks without installing anything. For the first time, “click here to reproduce my analysis” became a realistic offer rather than a rhetorical one.

The honest assessment is that these tools made reproducibility *achievable* but not *automatic*, and each has limits worth stating plainly. Public Binder sessions are ephemeral and resource-limited, unsuited to long or data-heavy computations. Container images pin software but not always the data, the hardware, or sources of nondeterminism such as random seeds, thread counts, and floating-point reduction order. Dependency specifications drift: an environment that omits exact versions may resolve differently months later, and upstream packages disappear. Reproducibility, in short, is a property a researcher must actively engineer (*e.g.*, by pinning versions, fixing seeds (as this notebook does in its setup cell), separating data acquisition from analysis, and testing that the pipeline runs end to end). Openness lowers the cost of doing so dramatically, because open formats and open infrastructure let the community build the supporting tools; but it does not remove the responsibility. The next section quantifies how often, in practice, that responsibility goes unmet.

7. The challenges of traditional notebooks

The very features that make notebooks excellent for exploration create difficulties when the same documents are asked to serve as durable, shareable software. The root cause is *hidden state*. A kernel holds a persistent namespace, and cells may be executed in any order and any number of times. The visible document records the cells and their last outputs, but not the sequence of executions that produced them. A notebook can therefore look correct on screen while depending on a variable defined in a cell that has since been deleted, or on a definition that no longer exists in the order shown (a discrepancy invisible until someone restarts the kernel and runs from the top).

The challenges of hidden state have drawn pointed criticism (most famously in Joel Grus’s 2018 JupyterCon talk, ‘I Don’t Like Notebooks’ [34]) and the empirical literature documents how widespread the consequences are. In the largest study of its kind, J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire [35] collected about 1.4 million Jupyter notebooks from roughly 264,000 GitHub repositories and attempted to execute the 863,878 that were valid Python with an unambiguous cell order. Only 24.1% ran to completion without raising an error, and only 4.0% reproduced the outputs stored in the document; failures were dominated by unmet dependencies (`ImportError`/`ModuleNotFoundError` accounted for about 29% of them). Symptoms of hidden state were pervasive: 36.4% of notebooks had been executed out of order, and 76.9% had at least one gap in their execution counters, consistent with cells that were re-run out of order or removed after execution. A follow-up that re-ran the analysis on a deduplicated corpus reached broadly similar conclusions [36]. Figure 7 summarizes the funnel.

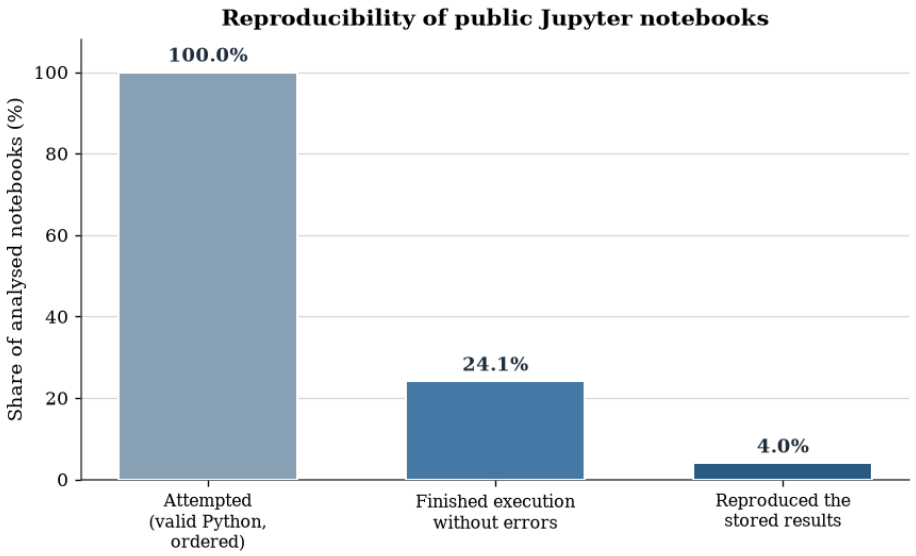


Figure 7. *The reproducibility funnel from J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire [35], as a share of the 863,878 valid notebooks they attempted to execute. The collapse from "runs without error" to "reproduces the stored results" measures the gap between a notebook that looks finished and one that actually is.*

The problems extend beyond execution. Because `.ipynb` files are JSON documents that embed outputs (including rendered images as base64 text), they interact badly with line-oriented version control: diffs are noisy and merges conflict over machine-generated metadata, discouraging the branching-and-review workflow that underpins collaborative software. Testing and debugging are awkward when the unit of code is a cell rather than an importable function, and a long notebook tends to accumulate globals and implicit ordering that resist refactoring as the analysis grows. Qualitative studies trace these frictions to a deeper tension. A. Rule, A. Tabard, and J. D. Hollan [37], analyzing over a million notebooks alongside interviews, found a persistent conflict between *exploration* and *explanation*: notebooks accrete exploratory mess and are seldom cleaned up for sharing. Interview and survey studies catalogue the resulting pain points across the whole workflow (*e.g.,* setup, exploration, refactoring, deployment, and collaboration) [38], and characterize the informal versioning and narrative practices analysts improvise in their absence [39]. A line of tools has tried to help from within: "code gathering" to extract the minimal cells behind a result [40], and methods to restore a reproducible execution order automatically [41], [42]. These are valuable, but they treat symptoms. The structural response (*i.e.,* bringing the notebook the engineering machinery that scripts and IDEs have long enjoyed) is the subject of the next section.

8. Open source IDE-native notebooks

The most important recent development in interactive scientific computing is a change in where notebooks live. This convergence of the exploratory notebook and the engineering workbench first took root with JupyterLab (architected by project leaders like Afshin Darian, Steven Silvester, and Paul Ivanov), which brought terminals, text editors, and notebooks into a single extensible interface. Today, the notebook is increasingly a first-class citizen inside a full software-development environment, sitting beside source files, a debugger, a test runner, a version-control panel, and a terminal, and benefiting from the same language intelligence as ordinary code. This is less a product category than a *movement* (a convergence of the exploratory notebook and the engineering workbench) and, like the developments before it, it was made possible by open standards.

Two protocols are central. The Language Server Protocol (LSP), opened by Microsoft in 2016 in collaboration with Red Hat and Codenvy, is a JSON-RPC standard by which an editor obtains language intelligence (*e.g.,* completion, hover documentation, go-to-definition, diagnostics, and refactorings) from a separate *language server* [43]. Its significance is combinatorial: instead of every one of M editors implementing support for every one of N languages, each editor and each language implement the protocol once, turning an $M \times N$ problem into an $M+N$ one. The Debug Adapter Protocol (DAP) does the same for debuggers [44]. Jupyter adopted both ideas. The Jupyter debugger protocol (JEP 47) [45] wraps DAP messages inside Jupyter messages (*e.g.,* `debug_request` and `debug_reply` on the Control channel, `debug_event` on IOPub) so that a frontend can drive breakpoints and variable inspection in a running kernel; the kernel advertises support in its `kernel_info_reply`, and implementations arrived through `debugpy` in `ipykernel` and through `xeus`-based kernels [45]. Because these protocols are open, a notebook hosted in any conforming environment can offer the same completion, refactoring, linting, and step-through debugging that script authors take for granted.

Open standards also reconciled notebooks with version control. Jupytext [46] represents a notebook as a plain-text script or Markdown file and keeps the two paired, so that the text form (free of outputs and volatile metadata) produces clean line diffs and reviewable pull requests, while the `.ipynb` retains rich outputs for reading. nbdime supplies content-aware diff and merge that understand notebook structure rather than raw JSON [47]. For execution and testing, papermill parameterizes and runs notebooks as pipeline stages [48]; nbval [49] turns a notebook into a regression test by comparing recomputed outputs against saved ones, and testbook unit-tests functions defined in notebooks directly [50]; and jupyter-cache re-executes only the cells that changed [51]. Perhaps the fullest expression of the convergence is nbdev [52], which treats notebooks as the source of truth for a Python package (*e.g.,* generating modules, tests, and documentation from them) and thereby realizes Knuth's literate-programming ideal four decades on [7], with the fast.ai library itself built this way [53].

Surrounding these notebook-aware tools is the ordinary furniture of a development environment, now within reach of the cell a scientist is editing. An integrated terminal makes shell commands (*e.g.,* installing a package, inspecting a data file, launching a job) part of the same session rather than a switch to another window. Package and environment management, whether through virtual environments, conda, or lockfile-based resolvers, can be driven from the same interface that runs the notebook, so that the environment a result depends on is created and recorded where the result is produced. Linters and type checkers flag mistakes as code is typed, and the plain-text representations that jupytext maintains drop directly into continuous-integration pipelines, where nbval or papermill can execute a notebook on every commit and fail the build if a figure or a number changes unexpectedly. None of these capabilities is new to software engineering; what is new is that the scientist need not leave the exploratory document to use any of them. Figure 8 places these capabilities against earlier generations.

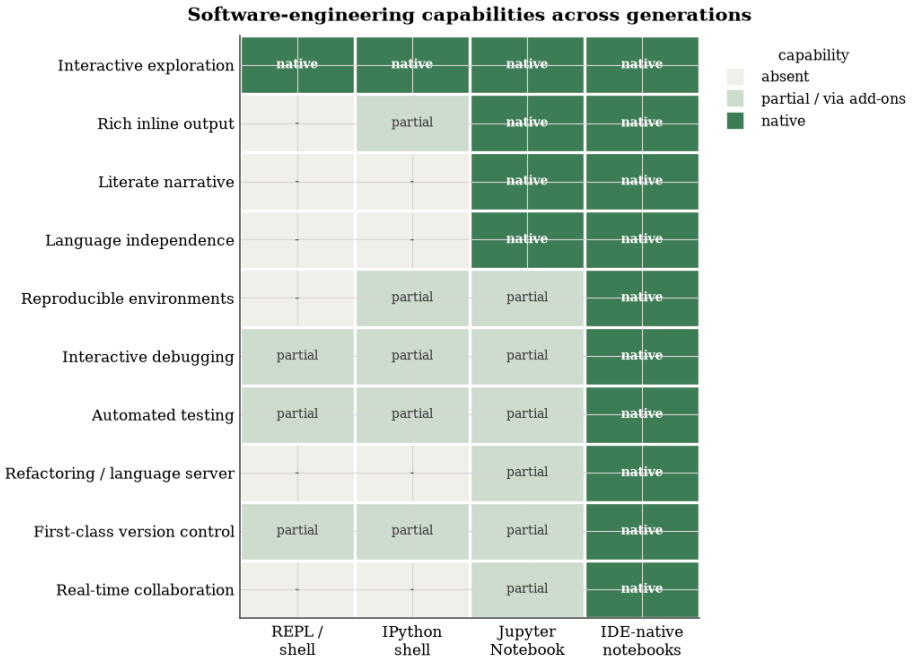


Figure 8. *A qualitative assessment by the authors of where ten software-engineering capabilities sit across four generations of interactive tools. The pattern, not any single cell, is the point: capabilities that classic notebooks could reach only through add-ons become native when the notebook is hosted inside a development environment (without sacrificing the interactive exploration of the first row).*

The decisive question is whether this engineering rigor comes at the expense of the exploratory freedom that made notebooks valuable. We argue that it need not, because the integration is *additive*. A scientist still opens a cell, types an idea, and runs it; what changes is that the same idea can, without leaving the environment, be linted as it is written, extracted into a tested function, stepped through in a debugger when it misbehaves, committed with a readable diff, and packaged for reuse. The exploratory loop is preserved; the cliff that once separated a promising notebook from maintainable software is graded into a ramp. The demonstration below enacts the smallest version of this transition. We write a function to a source module (so that notebook and script literally share the same code) then import it back and test it against an analytic solution, all without leaving the document.

```
%%writefile cooling.py
# A small module written from the notebook -- source and notebook now coexist.
import numpy as np

def newton_cooling(T0, T_env, k, t):
    """Analytic solution of Newton's law of cooling: T(t)=T_env+(T0-T_env)e^{-kt}."""
    return T_env + (T0 - T_env) * np.exp(-k * np.asarray(t, dtype=float))

def euler_step(T, T_env, k, dt):
    """One explicit-Euler step of dT/dt = -k (T - T_env)."""
    return T + dt * (-k * (T - T_env))
```

```
Writing cooling.py
```

```
import numpy as np
from cooling import newton_cooling, euler_step      # the module we just wrote

# A lightweight in-notebook test: numerical integration must approach the
# analytic solution as the step size shrinks. This is the habit that nbval,
# testbook, and CI build upon -- testable code, defined and checked in place.
def integrate(T0, T_env, k, t_end, dt):
    T, ts, Ts = T0, [0.0], [T0]
    while ts[-1] < t_end - 1e-9:
        T = euler_step(T, T_env, k, dt); ts.append(ts[-1] + dt); Ts.append(T)
    return np.array(ts), np.array(Ts)

t_end = 5.0
errors = {}
for dt in (0.5, 0.1, 0.01):
    ts, Ts = integrate(90.0, 20.0, 0.7, t_end, dt)
    errors[dt] = np.max(np.abs(Ts - newton_cooling(90.0, 20.0, 0.7, ts)))

assert errors[0.5] > errors[0.1] > errors[0.01], "error must shrink with dt"
assert errors[0.01] < 0.5, "fine step should track the analytic solution"
print("test passed -- max error by step size:",
      {dt: round(e, 3) for dt, e in errors.items()})
```

```
test passed -- max error by step size: {0.5: np.float64(5.272), 0.1: np.float64(0.929), 0.01:
np.float64(0.09)}
```

That cell is mundane on purpose. A function lives in a real `.py` module *and* in the running notebook; a test asserts a mathematical property and would fail loudly if the implementation regressed; and the whole exchange happened inside the exploratory document. Multiply this by language-server completion, a visual debugger, and a version-control panel, and the notebook stops being a place from which code must eventually escape to "become serious," and becomes a place where serious software can be written from the start.

9. Artificial Intelligence in the Notebook

The emergence of large language models represents the latest shift in interactive scientific computing. Unlike earlier advances that primarily changed interfaces or execution environments, AI assistants increasingly participate directly in the process of exploration by generating code, explaining results, suggesting analyses, and helping users navigate complex software ecosystems. Early work on neural code generation (including systems such as Codex, a GPT model fine-tuned on public source code [54]) demonstrated that models could synthesize useful programs from natural language, and subsequent systems have extended these capabilities into notebook environments where they complete lines of code, draft entire cells, answer questions about datasets, and edit notebooks through natural-language interaction.

Like earlier developments in the history of interactive computing, these systems further shorten the path from an idea to executable code. At the same time, empirical studies suggest a more nuanced picture than simple productivity gains. A controlled study of an LLM-based code completion tool found that it did not consistently reduce task completion time and that programmers often invested substantial effort in understanding and verifying generated code [55]. Observational work identified two distinct modes of interaction (*acceleration*, in which developers use the model to generate code they already intend to write, and *exploration*, in which the model helps discover unfamiliar approaches), each with different strengths and failure modes [56]. More recently, research has begun examining how AI assistants should be designed specifically for notebooks, where code, outputs, visualizations, and narrative coexist within a single computational document [57].

For scientific computing, the opportunities must be considered alongside longstanding concerns about reproducibility, transparency, and trust. As AI systems become collaborators rather than mere autocomplete tools, they complicate the notion of *provenance*. If portions of an analysis originate from prompts, the scientific record must capture what was asked, produced, and ultimately verified by the human.

Notebooks are particularly well suited to supporting this richer notion of provenance because they already interleave narrative, code, outputs, and execution history within a single computational artifact. Previous work on automatic provenance capture in notebooks [58], [59] suggests that AI generation events could likewise become part of the scientific record rather than remaining implicit. From this perspective, AI does not eliminate the need for careful scientific practice; instead, it shifts the balance of effort from writing code toward evaluating, validating, and documenting computational reasoning. As interactive scientific computing continues to evolve, openness, reproducibility, and transparent provenance become increasingly important (not despite AI, but because human accountability remains essential regardless of how code is produced).

In a world where some code is written by people and some by models, a verifiable trail of *what was computed, with what, and by whom* is the form of trust that openness uniquely enables. Human oversight remains the load-bearing element; the tooling exists to make that oversight cheap and auditable rather than to replace it.

10. Future directions

The next stage of interactive scientific computing may be defined less by faster computation than by richer interaction. Over the past twenty-five years, notebooks have gradually dissolved boundaries between exploration, documentation, publication, and software development. The next decade is likely to dissolve another boundary: the distinction between computational artifacts and interactive applications.

Notebook environments are evolving beyond static computational documents. Reactive execution models, richer user interface components, and browser-native runtimes allow notebooks to become scientific applications that users can interact with directly rather than merely execute. Instead of asking readers to modify source code, authors can increasingly expose parameters through forms, sliders, tables, and visual controls that invite exploration without requiring programming expertise. In this sense, notebooks are becoming environments not only for writing analyses but also for communicating and sharing them with broader audiences.

Whether this future benefits science will depend, once again, on openness. Interactive documents, reactive notebook systems, browser-based execution, and AI-assisted workflows all depend on open document formats, open protocols, interoperable runtimes, and transparent governance. The history traced throughout this paper repeatedly demonstrates that

the greatest advances in interactive scientific computing have emerged not from proprietary lock-in but from communities that chose openness at critical moments. Preserving that tradition offers the strongest foundation for the next generation of interactive scientific computing.

11. Conclusion

The history of interactive scientific computing is the history of reducing friction across four fundamental activities: exploratory analysis, software engineering, collaboration, and reproducible publication. As we have seen, each generation of tools inherited the strengths of its predecessors while internalizing capabilities that previously required separate, disjoint environments. The transition from REPLs to the IPython shell brought rich exploration and the foundations of a scientific ecosystem. Project Jupyter decoupled the interface from the runtime, creating a language-agnostic protocol that made the notebook a universal medium for literate narrative and reproducible environments. Most recently, the embedding of notebooks within IDEs has restored the rigorous software engineering practices (*e.g.,* debugging, testing, and version control) that early notebooks struggled to support.

Throughout this evolution, openness has been the decisive mechanism. Open source code, open community governance, and (crucially) open standards like the Jupyter messaging protocol and the `nbformat` schema [17] were the enablers that allowed these tools to scale from individual scripts to a ubiquitous scientific infrastructure. As the community continues to integrate new paradigms, such as AI-assisted code generation and reactive publishing, maintaining this commitment to transparent, reproducible, and open scientific computing remains as essential as ever.

Acknowledgements

This paper stands on the work of the IPython and Project Jupyter contributors, the maintainers of the scientific Python ecosystem, and the researchers whose empirical studies of notebooks are cited throughout. The factual claims herein were checked against primary sources during preparation; any remaining errors are the author's own.

Portions of this paper's text and code were drafted and debugged with the assistance of Claude (Anthropic). All AI-assisted output was reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content.

References

[1] D. L. Donoho, "An invitation to reproducible computational research," *Biostatistics*, vol. 11, no. 3, pp. 385–388, 2010, doi: 10.1093/biostatistics/kxq028.

[2] J. F. Claerbout and M. Karrenbach, "Electronic Documents Give Reproducible Research a New Meaning," in *SEG Technical Program Expanded Abstracts 1992*, 1992, pp. 601–604. doi: 10.1190/1.1822162.

[3] J. B. Buckheit and D. L. Donoho, "WaveLab and Reproducible Research," in *Wavelets and Statistics*, vol. 103, A. Antoniadis and G. Oppenheim, Eds., in Lecture Notes in Statistics, vol. 103., New York, NY: Springer, 1995, pp. 55–81. doi: 10.1007/978-1-4612-2544-7_5.

[4] V. Stodden *et al.*, "Enhancing reproducibility for computational methods," *Science*, vol. 354, no. 6317, pp. 1240–1241, 2016, doi: 10.1126/science.aah6168.

[5] G. Wilson *et al.*, "Best Practices for Scientific Computing," *PLoS Biology*, vol. 12, no. 1, p. e1001745, 2014, doi: 10.1371/journal.pbio.1001745.

[6] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal, "Good enough practices in scientific computing," *PLOS Computational Biology*, vol. 13, no. 6, p. e1005510, 2017, doi: 10.1371/journal.pcbi.1005510.

[7] D. E. Knuth, "Literate Programming," *The Computer Journal*, vol. 27, no. 2, pp. 97–111, 1984, doi: 10.1093/comjnl/27.2.97.

[8] C. Moler and J. Little, "A History of MATLAB," *Proceedings of the ACM on Programming Languages*, vol. 4, pp. 1–67, 2020, doi: 10.1145/3386331.

[9] R. A. Becker and J. M. Chambers, *S: An Interactive Environment for Data Analysis and Graphics*. Belmont, CA: Wadsworth, 1984.

[10] R. Ihaka and R. Gentleman, "R: A Language for Data Analysis and Graphics," *Journal of Computational and Graphical Statistics*, vol. 5, no. 3, pp. 299–314, 1996, doi: 10.1080/10618600.1996.10474713.

[11] D. E. Knuth, *Literate Programming*, no. 27. in CSLI Lecture Notes. Stanford, CA: Center for the Study of Language, 1992, p. xvi+368.

[12] F. Pérez, "IPython 0.0.1: a simple script to be loaded as $PYTHONSTARTUP, of historical interest only." [Online]. Available: https://gist.github.com/fperez/1579699

[13] F. Pérez and B. E. Granger, "IPython: A System for Interactive Scientific Computing," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 21–29, 2007, doi: 10.1109/MCSE.2007.53.

[14] J. M. Perkel, "Why Jupyter is Data Scientists' Computational Notebook of Choice," *Nature*, vol. 563, no. 7729, pp. 145–146, 2018, doi: 10.1038/d41586-018-07196-1.

[15] T. Kluyver *et al.*, "Jupyter Notebooks—a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas. Proceedings of the 20th International Conference on Electronic Publishing (ELPUB)*, F. Loizides and B. Schmidt, Eds., 2016, pp. 87–90. doi: 10.3233/978-1-61499-649-1-87.

[16] Jupyter Development Team, "Messaging in Jupyter (Jupyter Client Messaging Specification)." [Online]. Available: https://jupyter-client.readthedocs.io/en/stable/messaging.html

[17] Jupyter Development Team, "The Notebook File Format (nbformat)." [Online]. Available: https://nbformat.readthedocs.io/en/latest/format_description.html

[18] B. E. Granger and F. Pérez, "Jupyter: Thinking and Storytelling With Code and Data," *Computing in Science & Engineering*, vol. 23, no. 2, pp. 7–14, 2021, doi: 10.1109/MCSE.2021.3059263.

[19] C. R. Harris *et al.*, "Array Programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[20] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[21] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[22] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[23] A. Meurer *et al.*, "SymPy: Symbolic Computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, 2017, doi: 10.7717/peerj-cs.103.

[24] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] S. Hoyer and J. Hamman, "xarray: N-D Labeled Arrays and Datasets in Python," *Journal of Open Research Software*, vol. 5, no. 1, p. 10, 2017, doi: 10.5334/jors.148.

[26] M. Rocklin, "Dask: Parallel Computation with Blocked Algorithms and Task Scheduling," in *Proceedings of the 14th Python in Science Conference*, K. Huff and J. Bergstra, Eds., 2015, pp. 126–132. doi: 10.25080/Majora-7b98e3ed-013.

[27] M. D. Wilkinson *et al.*, "The FAIR Guiding Principles for scientific data management and stewardship," *Scientific Data*, vol. 3, no. 1, p. 160018, 2016, doi: 10.1038/sdata.2016.18.

[28] M. Barker *et al.*, "Introducing the FAIR Principles for research software," *Scientific Data*, vol. 9, no. 1, p. 622, 2022, doi: 10.1038/s41597-022-01710-x.

[29] J. B. Hamrick, C. Kelty, and U. Berkeley, "nbgrader: A Tool for Creating and Grading Assignments in the Jupyter Notebook," in *Proceedings of the 15th Python in Science Conference*, 2016, pp. 23–29. doi: 10.25080/Majora-629e541a-004.

[30] L. A. Barba, "Computational Thinking and Thinking About Computing," *IEEE Computing in Science & Engineering*, vol. 18, no. 4, pp. 65–66, 2016, doi: 10.1109/MCSE.2016.63.

[31] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten Simple Rules for Reproducible Computational Research," *PLoS Computational Biology*, vol. 9, no. 10, p. e1003285, 2013, doi: 10.1371/journal.pcbi.1003285.

[32] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.

[33] Project Jupyter *et al.*, "Binder 2.0 — Reproducible, Interactive, Sharable Environments for Science at Scale," in *Proceedings of the 17th Python in Science Conference*, F. Akici, D. Lippa, D. Niederhut, and M. Pacer, Eds., 2018, pp. 113–120. doi: 10.25080/Majora-4af1f417-011.

[34] J. Grus, “I Don't Like Notebooks.” [Online]. Available: https://docs.google.com/presentation/d/1n2RlMdmv1p25Xy5thJUhkKGvjtV-dkAIsUXP-AL4ffI/

[35] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “A Large-Scale Study about Quality and Reproducibility of Jupyter Notebooks,” in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, in MSR '19. Montreal, Quebec, Canada, 2019, pp. 507–517. doi: 10.1109/MSR.2019.00077.

[36] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “Understanding and Improving the Quality and Reproducibility of Jupyter Notebooks,” *Empirical Software Engineering*, vol. 26, no. 4, 2021, doi: 10.1007/s10664-021-09961-9.

[37] A. Rule, A. Tabard, and J. D. Hollan, “Exploration and Explanation in Computational Notebooks,” in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, in CHI '18. New York, NY, USA, 2018, pp. 1–12. doi: 10.1145/3173574.3173606.

[38] S. Chattopadhyay, I. Prasad, A. Z. Henley, A. Sarma, and T. Barik, “What's Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities,” in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, in CHI '20. New York, NY, USA, 2020, pp. 1–12. doi: 10.1145/3313831.3376729.

[39] M. B. Kery, M. Radensky, M. Arya, B. E. John, and B. A. Myers, “The Story in the Notebook: Exploratory Data Science using a Literate Programming Tool,” in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, in CHI '18. New York, NY, USA, 2018, pp. 1–11. doi: 10.1145/3173574.3173748.

[40] A. Head, F. Hohman, T. Barik, S. M. Drucker, and R. DeLine, “Managing Messes in Computational Notebooks,” in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, in CHI '19. New York, NY, USA, 2019, pp. 1–12. doi: 10.1145/3290605.3300500.

[41] J. Wang, L. Li, and A. Zeller, “Better Code, Better Sharing: On the Need of Analyzing Jupyter Notebooks,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, in ICSE-NIER '20. New York, NY, USA, 2020, pp. 53–56. doi: 10.1145/3377816.3381724.

[42] J. Wang, T.-Y. Li, L. Li, and A. Zeller, “Restoring Reproducibility of Jupyter Notebooks,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, in ICSE '20. New York, NY, USA, 2020, pp. 288–289. doi: 10.1145/3377812.3390803.

[43] Microsoft, “Language Server Protocol Specification.” [Online]. Available: https://microsoft.github.io/language-server-protocol/

[44] Microsoft, “Debug Adapter Protocol.” [Online]. Available: https://microsoft.github.io/debug-adapter-protocol/

[45] S. Corlay and J. Mabille, “Jupyter Enhancement Proposal 47: Jupyter Debugger Protocol.” [Online]. Available: https://jupyter.org/enhancement-proposals/47-jupyter-debugger-protocol/jupyter-debugger-protocol.html

[46] M. Wouts, “Jupytext: Jupyter Notebooks as Markdown Documents, Julia, Python or R Scripts.” [Online]. Available: https://github.com/mwouts/jupytext

[47] Project Jupyter, “nbdime: Diffing and Merging of Jupyter Notebooks.” [Online]. Available: https://github.com/jupyter/nbdime

[48] nteract contributors, “papermill: Parameterize, Execute, and Analyze Notebooks.” [Online]. Available: https://github.com/nteract/papermill

[49] H. Fangohr *et al.*, “Testing with Jupyter Notebooks: NoteBook VALidation (nbval) Plug-in for pytest.” [Online]. Available: https://arxiv.org/abs/2001.04808

[50] nteract contributors, “testbook: Unit Testing for Jupyter Notebooks.” [Online]. Available: https://github.com/nteract/testbook

[51] Executable Books Project, “jupyter-cache: A Defined Interface for Working with a Cache of Executed Jupyter Notebooks.” [Online]. Available: https://github.com/executablebooks/jupyter-cache

[52] J. Howard and H. Husain, “nbdev: Create Delightful Software with Jupyter Notebooks.” [Online]. Available: https://github.com/fastai/nbdev

[53] J. Howard and S. Gugger, “fastai: A Layered API for Deep Learning,” *Information*, vol. 11, no. 2, p. 108, 2020, doi: 10.3390/info11020108.

[54] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021, doi: 10.48550/arXiv.2107.03374.

[55] P. Vaithilingam, T. Zhang, and E. L. Glassman, “Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models,” in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, in CHI EA '22. New York, NY, USA, 2022, pp. 1–7. doi: 10.1145/3491101.3519665.

[56] S. Barke, M. B. James, and N. Polikarpova, “Grounded Copilot: How Programmers Interact with Code-Generating Models,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 85–111, 2023, doi: 10.1145/3586030.

[57] A. M. McNutt, C. Wang, R. A. DeLine, and S. M. Drucker, “On the Design of AI-powered Code Assistants for Notebooks,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, in CHI '23. New York, NY, USA, 2023, pp. 1–16. doi: 10.1145/3544548.3580940.

[58] J. F. Pimentel, V. Braganholo, L. Murta, and J. Freire, “Collecting and Analyzing Provenance on Interactive Notebooks: When IPython Meets noWorkflow,” in *7th USENIX Workshop on the Theory and Practice of Provenance (TaPP 2015)*, Edinburgh, Scotland, 2015. [Online]. Available: https://www.usenix.org/conference/tapp15/workshop-program/presentation/pimentel

[59] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “noWorkflow: a Tool for Collecting, Analyzing, and Managing Provenance from Python Scripts,” *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1841–1844, 2017, doi: 10.14778/3137765.3137789.